PROJECT GUIDE

Ghana Hospital Management System

What it does, how it is built, and where it stands today.

Prepared 24 September 2026

Reflects the codebase as checked on that date: 83 backend and 50 frontend automated tests passing.

Stack: React + TypeScript, Django REST Framework, PostgreSQL.

Development uses synthetic demo data only; the project holds no real patient information.

Contents

1. The project at a glance . . . 3

2. Who uses it, and how a visit flows . . . 4

3. Architecture . . . 5

4. The backend, app by app . . . 6

5. The data model . . . 7

6. Access control (RBAC) . . . 8

7. Security, privacy and the audit log . . . 9

8. Built for Ghana . . . 10

9. The frontend . . . 11

10. Protecting patients: the adversarial cases . . . 12

11. Testing and verification . . . 12

12. How it got here: decisions and history . . . 13

13. Running it yourself . . . 13

14. Where it stands and what comes next . . . 14

HOW TO READ THIS GUIDE

Sections 1 to 3 explain what the system is and how a patient visit flows through it; that is enough for a general understanding. Sections 4 to 10 go deeper into the design and are the ones to use when someone asks "how does it actually work?". Sections 11 to 14 cover testing, history, running it yourself, and what is left to do. Where something is unfinished, unverified, or a deliberate compromise, this guide says so plainly.

1. The project at a glance

The Ghana Hospital Management System (HMS) is a web application that lets a hospital or clinic run a patient's visit on one system, from the front desk to the consulting room, the laboratory, the pharmacy and the ward. Each staff member signs in with their own account and only sees the screens their role allows. The rules behind that are enforced by the server, not merely hidden in the interface.

It was designed for Ghanaian hospitals first. Ghana Card numbers, Ghanaian mobile numbers, the Africa/Accra time zone and NHIS-style insurance are built in as real concepts rather than afterthoughts, while avoiding Ghana-only assumptions wherever that costs little, so other countries could be supported later.

8
staff and patient roles

34
database tables (models)

14
backend apps, one per domain

133
automated tests passing (83 backend + 50 frontend)

What it covers today

- Patient registration, search, insurance and allergies
- Appointment booking and cancellation
- Queue and check-in, with emergency priority
- Vital signs recorded by nurses
- Doctor consultation, diagnosis and medical history
- Laboratory orders, results and validation
- Pharmacy prescriptions and stock-checked dispensing
- Inventory: items, suppliers, stock movements
- Admissions: wards, rooms, beds, transfer, discharge
- Referrals, internal and external
- User accounts, roles and an append-only audit log
- A role-aware dashboard with live figures

What it deliberately does not include

- **Billing and payments.** A full Billing module (invoices, cash and Mobile Money payments) was built in Phase 4 and then removed on purpose on 2026-09-08 at your request, together with the Cashier role.
- **SMS or email notifications.** The interface is designed, but no notification code exists yet.
- **Reports and analytics.** Planned for Phase 5; only a small live dashboard exists today.
- **A patient portal.** The Patient role exists but holds no permissions until that portal is built.

WHERE IT STANDS

Phases 0 to 4 are built, and each was verified with a live run through the real API, not just by reading code. Phase 5 (reports, security hardening, browser end-to-end tests) has not started. One thing to know: the Git repository has no commits yet, so none of this work has a saved history.

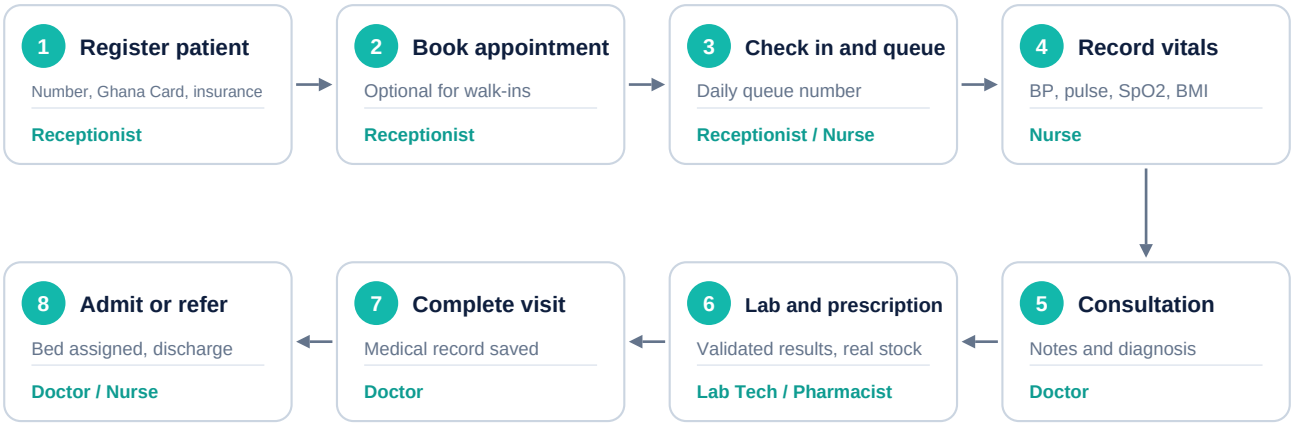
2. Who uses it, and how a visit flows

There are eight roles. Seven are for hospital staff; the eighth, Patient, is reserved for a future portal.

Role	What they do in the system
System Administrator	Platform-level: manages user accounts and reads the audit log. Deliberately has no access to patient or clinical data and is not attached to a hospital.
Hospital Administrator	Oversees one hospital: manages staff accounts, views patients, appointments, queue, labs and pharmacy, manages the inventory catalogue, and sets up wards, rooms and beds.
Doctor	Registers walk-in and emergency patients, books appointments, runs consultations, records diagnoses, orders lab tests, prescribes, admits patients, refers.
Nurse	Checks patients in and manages the queue, records vital signs and allergies, orders lab tests, admits patients.
Receptionist	Registers patients, books and cancels appointments, checks patients in, manages the queue, and can delete or restore a patient record.
Laboratory Technician	Collects samples, enters and validates results, records movements of lab supplies.
Pharmacist	Sees prescriptions and dispenses them against real stock; records stock movements.
Patient	Reserved for the future patient portal. Holds no permissions today.

A patient's journey through the system

The diagram shows the main path. Each numbered step is a real screen or action in the app; the table below it explains what happens and what the system refuses to allow at each step.

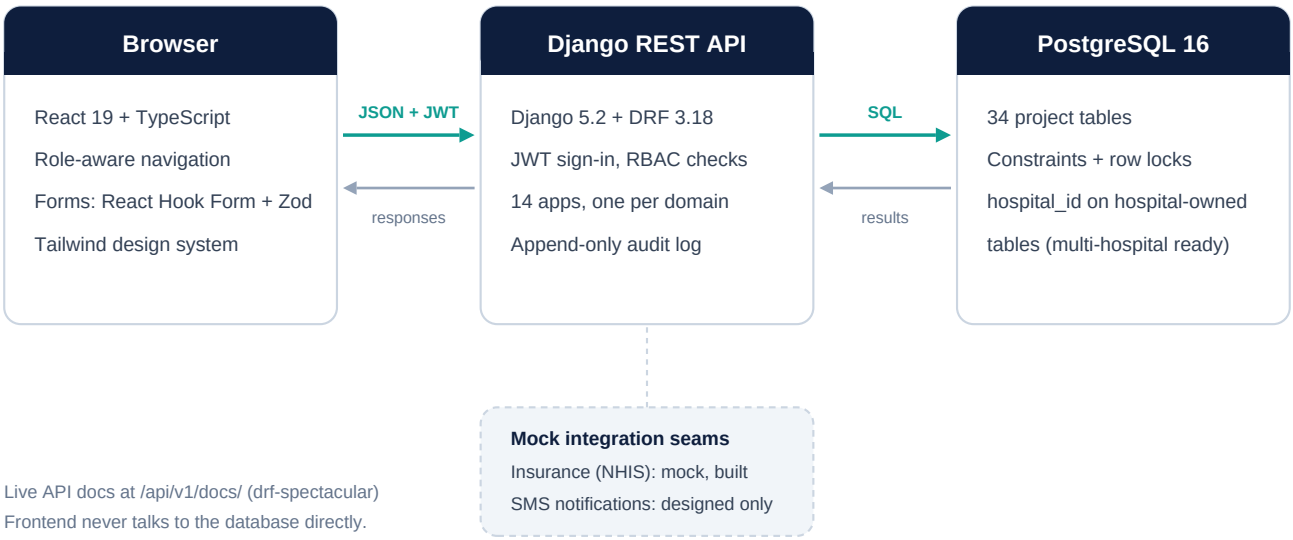


Step	What happens	What the system enforces
1. Register	Reception enters the patient's details, Ghana Card number, phone and insurance. A patient number such as PAT-000001 is generated.	Ghana Card must match GHA-XXXXXXXXX-X. Phones are normalised to +233 format. The same Ghana Card cannot be registered twice at one hospital.
2. Book	A department, optional doctor, date and time, and reason are chosen.	Double-booking a doctor at the same time is rejected. Booking in the past is rejected. Patient and department must belong to your hospital. Cancelling never deletes the row.

Step	What happens	What the system enforces
3. Check in	A Visit and a queue entry are created together; a linked appointment becomes "checked in".	Queue numbers restart each day per department. Emergencies are ordered first. A patient cannot have two active visits, and a deleted patient cannot be checked in.
4. Vitals	A nurse records temperature, blood pressure, pulse, respiratory rate, oxygen saturation, weight and height.	BMI is calculated by the server and never accepted from the browser. More than one reading per visit is allowed.
5. Consultation	The doctor starts a consultation (the queue entry moves to "in consultation"), records notes and adds diagnoses.	One consultation per visit. At most one primary diagnosis. Edits are only possible while it is in progress.
6. Lab	The doctor orders tests. A lab technician collects the sample, enters a result, then validates it.	A result cannot be entered before the sample is collected. Until validated, the value is hidden from every role except the lab technician.
7. Prescription	The doctor prescribes; the pharmacist dispenses.	Dispensing cannot exceed what was prescribed or what is in stock (checked under a database lock). Every dispense writes a stock transaction.
8. Complete	The doctor completes the consultation. A permanent medical-record snapshot is saved, the visit is closed, and the queue entry and appointment are completed.	Needs at least one diagnosis. All of it happens in one transaction, so none of it can end up half-done. The consultation is locked afterwards.
9. Admit or refer	A patient can be admitted to a specific bed, transferred, and later discharged with a summary. A referral to an internal or external destination can be raised.	A bed can hold only one active admission, enforced by the database itself. Transfers to an occupied bed are rejected. Referrals follow allowed status steps only.

3. Architecture

HMS is a **modular monolith**: one Django project, split into one app per area of the hospital, talking to one PostgreSQL database, and serving a separate React application through a versioned JSON API (`/api/v1/`). It was chosen because it is simple for a small team to run and understand, while the app boundaries keep the code organised enough to split up later if that ever becomes necessary.



Live API docs at /api/v1/docs/ (drf-spectacular)
Frontend never talks to the database directly.

Technology

Layer	What is used	Notes
Frontend	React 19, Vite 8, TypeScript 6, React Router 7, Tailwind CSS 4, Axios, React Hook Form + Zod, lucide-react icons	17 pages, one shared component library
Backend	Python, Django 5.2, Django REST Framework 3.18, SimpleJWT, drf-spectacular, django-cors-headers, Argon2	Versions pinned exactly in requirements.txt
Database	PostgreSQL 16	Runs natively via Homebrew on this Mac
Testing	pytest + pytest-django; Vitest + React Testing Library	133 tests
Packaging	Docker Compose (Postgres + backend + frontend)	Written but never run on this Mac; see below

How a request travels

- **Sign in.** Email and password go to the server, which returns a 15-minute access token and a 7-day refresh token. Passwords are stored hashed with Argon2.
- **Every call carries the access token.** The frontend keeps it in memory and attaches it to each API request.
- **Permission check.** The server looks up the user's role and checks it holds the needed action (view, create, edit, delete) on that module.
- **Hospital check.** Results are limited to the user's own hospital; a record from another hospital simply answers "not found".
- **The change is made and audited.** Important actions append a row to the audit log.
- **Errors are cleaned.** Outside debug mode, unexpected errors never leak internal details to the browser.

WHERE THE BUILD DIFFERS FROM THE ORIGINAL BRIEF

The brief suggested React Query for server data; the app uses Axios with ordinary component state instead. Playwright browser tests were planned but are not set up yet (Phase 5). The Docker Compose files are written but have never been run on this machine, because macOS 12 cannot run Docker Desktop; everything was verified with Postgres, Django and Vite running natively instead.

4. The backend, app by app

Each Django app owns one area of the hospital and its own tables, API routes, and tests.

App	Responsibility	Models
core	Hospital and Department, shared base classes, the RBAC permission class, the Ghana Card and phone validators, the cleaned-up error handler	Hospital, Department
users	Staff accounts (sign in with email), roles and permissions; the seed command that creates demo data	User, Role, Permission, RolePermission
authentication	Sign-in, token refresh and sign-out endpoints	(uses users tables)
audit	The append-only record of who did what and when	AuditLog
patients	Patient records, per-hospital patient numbers, allergies, search, soft delete and restore	Patient, PatientNumberSequence, Allergy

App	Responsibility	Models
insurance	NHIS and private insurance per patient, plus a mock eligibility check	Insurance
appointments	Booking, rescheduling, cancelling, double-booking prevention	Appointment
queues	Check-in, the visit (encounter), daily queue numbers, emergency priority	Visit, Queue, QueueNumberSequence
consultations	Vitals, the consultation, diagnoses, and the medical-record snapshot	VitalSign, Consultation, Diagnosis, MedicalRecord
laboratory	Test catalogue, lab requests, results and validation	LaboratoryTest, LaboratoryRequest, LaboratoryResult
pharmacy	Medication catalogue, prescriptions, dispensing	Medication, Prescription, PrescriptionItem
inventory	Stock items, suppliers, and every stock movement	Supplier, InventoryItem, InventoryTransaction
admissions	Wards, rooms, beds, admission, transfer, discharge	Ward, Room, Bed, Admission, Discharge
referrals	Internal and external referrals with status tracking	Referral

5. The data model

The most important idea in the database is that the **Visit** is the centre. A patient is a long-lived record; each time they come to the hospital, a Visit is created, and everything clinical from that trip (queue entry, vitals, consultation, lab tests, prescription, admission, referral) is attached to it. That is why an appointment and a visit are separate things: an appointment is only a booking, while walk-in patients still get a visit.



Prescription and lab requests hang off the Consultation, and the Discharge hangs off the Admission. Every hospital-owned table also carries a hospital_id, so supporting several hospitals later is a data task rather than a redesign.

Rules that apply everywhere

- **Two kinds of ID.** Anything patient-facing or visible in a URL uses an unguessable UUID, which also makes guessing other people's records impractical. Small internal lookup tables use simple numbers.
- **Nothing clinical is ever really deleted.** Patients, consultations, diagnoses, prescriptions, lab results and admissions are soft-deleted or status-changed, and the tables that refer to them are protected from cascading deletes.
- **Some tables are append-only.** The audit log, stock transactions and medical records have no update or delete path at all, not even for the System Administrator.
- **Reference data is deactivated, not removed.** Medications, inventory items, wards, rooms and beds are switched off with an active flag.

Design decisions worth knowing

- **Medical record is a snapshot.** The brief listed it without defining it. It became a permanent, read-only summary created automatically when a consultation is completed, so a patient's history never drifts from the working data.
- **Stock is a running total backed by a ledger.** An item's quantity on hand only ever changes by recording a stock transaction, so the total and the history cannot disagree.
- **Numbers are generated under a lock.** Patient numbers and daily queue numbers come from counter rows updated under a database lock, so two people registering at the same moment never receive the same number.
- **Medication and stock are separate.** Medication is the clinical catalogue; an inventory item is generic stock that may or may not be a medication, so bandages and gloves fit the same stock system.
- **Lab tests are a shared catalogue.** Unlike departments or medications, they are not owned by one hospital.

6. Access control (RBAC)

RBAC means role-based access control: what you can do depends on your role. The table below is generated from the seed data that creates these grants, so it matches what the system enforces. Anything not listed is denied.

Module	System Admin	Hospital Admin	Doctor	Nurse	Reception	Lab Tech	Pharmacist
Users & roles	VCED	VCE	-	-	-	-	-
Audit log	V	V	-	-	-	-	-
Patient records	-	VE	VC	VC	VCE	V	V
Appointments	-	V	VCE	V	VCED	-	-
Queue	-	V	V	VCE	VCE	V	V
Vitals	-	V	V	VCE	-	-	-
Consultation	-	V	VCE	V	-	-	V
Laboratory	-	V	VC	VC	-	VCE	-
Pharmacy	-	V	VC	V	-	-	VCE
Inventory	-	VCE	-	V	-	VE	VE
Admissions	-	VC	VC	VC	V	-	-
Referrals	-	V	VCE	V	V	-	-

V view, C create, E edit, D delete or deactivate; a dash means no access. The Patient role has no grants until a portal exists.

How it is enforced

- **On the server, always.** A single permission class checks the user's role against the module and action for every request. Hiding a menu item is only a convenience; the server would refuse the request anyway.
- **Default deny.** Every endpoint requires sign-in unless it is explicitly opened. Only the health check, sign-in and token refresh are open.
- **Role-aware navigation.** After sign-in, the server tells the app which modules the user may view. The sidebar shows only those, and a direct link to a forbidden page shows "Access restricted".
- **A live demonstration.** The dashboard has an "RBAC check" card whose button calls an admin-only endpoint. Admins see the staff list; everyone else gets a real 403 response, proving the server is the gatekeeper.
- **The role picker on the login page is a confirmation, not a credential.** If the chosen role does not match the account, the app signs the user straight back out and explains why.

Deliberate deviations from the drafted matrix

- Hospital Administrator was given *create* on Admissions so they can set up wards, rooms and beds, although admitting a patient is a clinical decision for doctors and nurses. This is flagged in the docs.
- Lab technicians and pharmacists hold *edit* but not *create* on Inventory, yet they record daily stock movements. That endpoint checks *edit* explicitly instead of the usual create rule.
- Starting a consultation moves the queue entry, and dispensing moves stock, as side effects of the doctor's or pharmacist's own action, so neither needs extra grants.

7. Security, privacy and the audit log

Ghana's Data Protection Act, 2012 (Act 843) is the compliance frame the design keeps in mind. This is design intent, not legal advice. Audit logging and access control are the parts of the system that carry it.

Control	How it works here
Password storage	Argon2 hashing, configured ahead of Django's default.
Sign-in tokens	15-minute access token, 7-day refresh token that rotates on every use, and is blacklisted on sign-out. A refresh token used after sign-out is rejected (tested).
Hospital isolation	Queries are filtered to the user's hospital. Fetching another hospital's record by guessed ID returns 404, not 403, so it does not even confirm the record exists. Writes that reference another hospital's patient or department are refused.
Audit log	Append-only. Records sign-ins (including failures), patient and appointment changes, check-ins, queue changes, consultations, diagnoses, medical-record access (reads, not just writes), lab activity, prescriptions, dispensing, admissions, transfers, discharges and referrals. Even the Django admin blocks editing or deleting it.
Business rules as controls	Allowed status transitions are listed explicitly (nothing can skip a step). Completed consultations are locked. Dispensing checks two limits. Bed double-booking is stopped by a database constraint, so even two simultaneous requests cannot both win.
Result confidentiality	A draft lab result is hidden from doctors and everyone else until a lab technician validates it.
Error hygiene	A custom handler strips internal detail from unexpected errors outside debug mode.
Secrets and CORS	Secrets come from a git-ignored .env file (only placeholders are in .env.example). Cross-origin access is limited to an explicit allow-list, verified with a real browser-style preflight request.

Known gaps and things still to do

These are listed so nobody is surprised later. None is hidden in the code; some were found while writing this guide.

- **No rate limiting yet** on sign-in attempts (planned for Phase 5).
- **No automatic refresh during a session.** The code renews the access token only when the page loads. Reading it, an open tab past the 15-minute lifetime would start getting 401 errors until reloaded. I have not reproduced this in a browser.
- **The refresh token is kept in the browser's localStorage.** A common trade-off, but it means a successful script-injection attack could read it. Rotation and blacklisting limit the damage; httpOnly cookies would be the stronger design.
- **Production hardening** (HTTPS redirect, HSTS, secure cookies) is not configured; Django's deployment check lists these as expected development-mode warnings.
- **The style-guide page** (/design-system) is public and not behind sign-in. It shows no data, but it should be removed or protected before release.
- **Hospital Administrators can read full clinical notes.** Whether they should see only metadata is an open question that was flagged, not silently decided.

8. Built for Ghana

A FIRM RULE ABOUT INTEGRATIONS

No code calls, or pretends to call, a real NHIS, Mobile Money or SMS provider. Anything speculative is a clearly labelled mock behind an interface, so a real integration can replace it later without changing the code that uses it.

Requirement	How it is handled	Status
Ghana Card	Stored in its own field and validated as GHA-XXXXXXXXX-X, separate from any other ID.	Built
Phone numbers	Accepts 024 123 4567 or +233 24 123 4567 and stores the international (E.164) form.	Built
Time zone	Africa/Accra throughout the backend, and in the dashboard's date and greeting.	Built
NHIS and private insurance	One generic Insurance record for either; a patient can hold several. Eligibility is checked through an interface backed by a **mock** provider.	Mock only, pending a real integration
Mobile Money and payments	A mock payment gateway with MTN, Telecel and AirtelTigo sub-types was built, then removed with Billing. The design remains valid if billing returns.	Removed
SMS notifications	A gateway interface with a console-log mock was designed so any provider can be swapped in.	Designed, not built
Currency	Ghana cedi stored as whole pesewas, never floating point. It existed for invoices and left with Billing.	Not in use today
Queue numbering	Restarts each day per department, matching how clinics announce "number 12 today".	Built
Low bandwidth	Every list is paginated (25 per page) and the interface is kept light.	Built; not measured

9. The frontend

The interface is a single-page React app. Everything after the login page is built from one shared component library (buttons, cards, tables, inputs, status pills, modals, skeleton loaders), so screens look and behave consistently. The palette is deep navy for structure with a teal accent for highlights, and status colours follow one rule everywhere: green means done or available, amber means waiting, red means stop or urgent.

Route	Screen	Needs access to
/login	Sign in with a role picker, email and password (with show/hide)	Public
/	Dashboard: greeting, live counts, today's appointments, RBAC check card	Any signed-in user
/patients, /patients/new	Search and register patients; a toggle shows deleted patients	Patients
/patients/:id	Profile, insurance and eligibility check, allergies, medical history, delete or restore	Patients
/appointments, /appointments/book	List and cancel; book by department and time	Appointments
/queue, /queue/check-in	Live queue board with department filter; check a patient in	Queue
/visits/:visitId	The visit workspace: vitals, consultation, lab, prescription, admission, referrals	Queue (each card needs its own module)
/laboratory	Lab technician worklist: collect, record, validate	Laboratory
/pharmacy	Pharmacist worklist: dispense against stock	Pharmacy
/inventory	Items, suppliers, stock movements, low-stock filter	Inventory
/admissions, /admissions/beds	Admissions board; ward, room and bed setup	Admissions
/referrals	Referrals worklist with status changes	Referrals
/design-system	Internal style guide (not linked, not protected)	Public

Two screens worth knowing

The visit workspace is the heart of the app. It opens from the queue and gathers the whole encounter on one page, with a card per stage. Each card loads on its own, and a role without access to one simply does not see it, rather than the whole page failing.

The dashboard shows only figures backed by live data: patients, today's appointments, people in the queue, beds occupied out of the total, and low-stock items. A card whose data the current role is not allowed to see is hidden rather than showing a wrong number. Figures such as revenue, doctor availability or a global search were left out on purpose, because nothing behind them exists.

10. Protecting patients: the adversarial cases

The brief asked for the system to be tested against people and situations that try to break it, not only the happy path. These are the cases covered by automated tests today.

Case	What the tests check
Unauthorised access	No token gives 401; a role without the grant gets 403 (tested across most modules).
IDOR (guessing IDs)	A staff member from another hospital cannot list or fetch that hospital's patients, visits or medical records, or create lab requests or allergies for its patients.
Double booking	The same doctor at the same time is rejected; the same bed twice is rejected; a transfer to an occupied bed is rejected.
Over-dispensing	Dispensing more than prescribed, or more than stock, is rejected; issuing more than stock is rejected.
Skipping steps	A lab request or referral cannot jump straight to completed; a result cannot be entered before the sample is collected.
Immutability	A completed consultation cannot be edited, get more diagnoses, or be started twice; a second primary diagnosis is rejected.
Bad data	Invalid Ghana Card or phone number, duplicate Ghana Card, booking in the past, or for a deleted patient.
Permission escalation	Roles lacking create, delete or edit cannot perform those actions, even though they can view (for example a pharmacist cannot add a catalogue item).

Cases tied to invoices and payment amounts were removed together with Billing. A complete, dedicated adversarial suite is still a Phase 5 goal.

11. Testing and verification

Check	Result on 24 September 2026
Backend tests (pytest-django)	83 passed
Frontend tests (Vitest + React Testing Library)	50 passed, in 14 files
Django system check	No issues
TypeScript type-check	Clean
Live end-to-end runs	At the end of each phase, the exit criterion was run against the real HTTP API on the dev servers (for example: check-in, vitals, consultation, diagnosis, complete, medical record visible; and prescription, dispense, stock decreased, over-dispense rejected).

What has not been verified

- **Full browser click-through of every screen.** No browser-automation tool was available, so the interface was verified through component tests, type-checking and builds, plus your own use of it.
- **Docker Compose.** Written but never run on this machine.
- **Playwright end-to-end tests.** Not yet set up.
- **Performance and load,** including behaviour on slow connections.

12. How it got here: decisions and history

When	What happened
2026-09-03	Planning approved. Locked the core rules: RBAC model, UUID versus number IDs, soft-delete policy, hospital scoping on every table, email as the staff login, and the three sanctioned mock integrations.
Phase 0	Scaffolding, then authentication: users, roles, JWT with rotation and blacklist, the first audit log, the RBAC deny path shown on the dashboard.
Phase 1	Patients (with Ghana Card and phone validation), insurance with a mock provider, appointments with double-booking prevention.
Phase 2	Queue and visits, vitals, consultation, diagnosis, allergies, and the medical-record snapshot.
Phase 3	Laboratory with two-step result validation, pharmacy with stock-checked dispensing, inventory with a transaction ledger.
Redesign	At your request, the frontend was rebuilt on Tailwind CSS with a shared component library and the navy and teal identity.
Phase 4	Billing, admissions and referrals. Bed double-booking became a database-level rule.
2026-09-08	Billing and the Cashier role were removed entirely at your explicit request: the app, its tables, its pricing fields and its screens. Cashier's audit history was kept by deactivating the account rather than deleting it.
2026-09-16	A Delete and Restore action for a single patient was added. It is a soft delete, and it was made to actually work: deleted patients vanish from search and cannot be checked in or booked until restored.
2026-09-23	The login page was redesigned with a role picker, and the dashboard restyled to a new layout using only real data. Items in the mockup with no real backing (revenue, doctor availability, emergency call button, search, notifications) were dropped by decision.

13. Running it yourself

Everything runs natively on this Mac (Postgres, Django and Vite), because Docker is not available on macOS 12. Run each block in its own terminal.

1. Database

```
brew services start postgresql@16
```

2. Backend (http://localhost:8000)

```
cd backend
source .venv/bin/activate
export DATABASE_URL="psql://hms:<password>@localhost:5432/hms"
python manage.py migrate
python manage.py seed_rbac # roles, demo accounts, sample catalogue
python manage.py runserver
```

3. Frontend (http://localhost:5173)

```
cd frontend
npm install
npm run dev
```

The `DATABASE_URL` override matters: the project's .env file points at the Docker hostname `db`, which does not exist when running natively. API documentation is at `/api/v1/docs/`. To run the tests: `pytest` in backend, `npm test` in frontend.

Demo accounts

The seed command creates one clearly fake account per role, all sharing the development-only password `DemoPass123!`. Pick the matching role on the login page.

Role	Email
System Administrator	system_admin@demo.hms.gh
Hospital Administrator	hospital_admin@demo.hms.gh
Doctor	doctor@demo.hms.gh
Nurse	nurse@demo.hms.gh
Receptionist	receptionist@demo.hms.gh
Laboratory Technician	lab_technician@demo.hms.gh
Pharmacist	pharmacist@demo.hms.gh

The seed also creates a demo hospital with two departments, a small lab-test and medication catalogue with starting stock, and one ward with two beds. It is safe to re-run.

14. Where it stands and what comes next

Phase	Scope	State
0	Scaffolding, authentication, RBAC, audit log	Done
1	Patients, insurance, appointments	Done
2	Queue, vitals, consultation, diagnosis, medical record	Done
3	Laboratory, pharmacy, inventory	Done
4	Admissions and referrals (Billing built, then removed)	Done
5	Reports, security hardening, end-to-end tests, accessibility audit, full adversarial suite	Not started

Suggested next steps

- **Make the first Git commit.** The repository has no history yet; everything is untracked.
- **Fix the session expiry gap** so a long-lived tab renews its token instead of failing.
- **Phase 5:** reporting views, rate limiting and security headers, Playwright tests of the full patient journey, and the accessibility audit.
- **Housekeeping:** docs/DEVELOPMENT.md still says the seed creates 9 roles (it is 8 since Cashier was removed), and the roadmap test counts trail the current 83 backend tests.

Open questions still on the table

These were raised during planning and deliberately not decided mid-build:

- How patient numbers should work across several future hospitals (currently unique per hospital).
- Whether the System Administrator needs an emergency break-glass path into clinical data.
- Whether Hospital Administrators should see full clinical notes or metadata only.
- Whether inventory access should be split by department or stay one shared module.
- How reporting should read data: database views or API-only.

Deliberately deferred

Architected for, but not built: a patient portal, a mobile app, real SMS, NHIS and Mobile Money integrations, offline mode, multi-hospital support, telemedicine, Power BI, and AI-assisted features.

A HARD RULE FOR ANY FUTURE AI FEATURE

Any AI capability is decision support only. It must never diagnose a patient on its own or replace a qualified healthcare professional's judgement.